

1. Mapa rápido para reconocer el patrón

No elijas por una palabra suelta. Elegí por la restricción que el sistema debe cumplir y confirmá que aparecen los participantes necesarios.

Si la consigna exige...	Patrón	Confirmación práctica
Cada subclase decide qué objeto concreto crear	Factory Method	Existe Creator base, método de creación redefinido y un solo tipo de Product.
Crear varios productos que deben pertenecer a la misma variante	Abstract Factory	Una fábrica elegida una vez crea toda la familia compatible.
Una sola instancia y acceso común controlado	Singleton	Constructor privado, instancia estática controlada y mismo objeto al pedirlo otra vez.
Estructura en árbol y misma operación para objeto y grupo	Composite	Hoja y compuesto comparten interfaz; el compuesto guarda hijos y delega recursivamente.
Un cambio debe avisar a varios receptores sin conocer sus clases	Observer	El sujeto mantiene suscriptores y los notifica mediante una interfaz.

MÉTODO DE LECTURA EN CUATRO PASOS

1. Subrayá la restricción. 2. Preguntá qué cambia: un producto, una familia, una cantidad de hijos o los receptores. 3. Dibujá la relación mínima. 4. Traducí cada rol a nombres del caso.

Descartes rápidos

- Una lista sola no demuestra Composite: tiene que haber estructura recursiva y trato uniforme.
- Muchos objetos no demuestran Observer: tiene que existir un evento y receptores suscriptos.
- Usar new no demuestra Factory Method: la creación debe redefinirse en subclases.
- Una configuración compartida no obliga a Singleton si no se pide controlar la unicidad.
- Varios productos no alcanzan para Abstract Factory: deben formar familias compatibles que no conviene mezclar.

FÓRMULA DE JUSTIFICACIÓN

"Elijo [patrón] porque la consigna exige [necesidad]. [clase] cumple el rol de [participante]. Mediante [mecanismo], logro [resultado observable]."

2. Factory Method

PISTA DECISIVA

Cada subclase creadora decide qué producto concreto instanciar. Cambia un producto; el flujo que lo usa queda estable.

Cuándo usarlo

- El algoritmo general es el mismo, pero el objeto que necesita cambia.
- La consigna menciona creadores concretos, subclases o redefinir la creación.
- La lógica cliente debe usar el contrato Product y no conocer cada clase concreta.

Cómo reconocer que NO es AF

- Aunque el objeto se llame kit, el enunciado lo trata como un producto.
- No se crean tres tipos coordinados; se elige una variante del mismo contrato.
- La frase decisiva es 'cada subclase decide cuál crear'.

Roles mínimos

Rol	En OmniExpress A
Product	IKitSeguridad
ConcreteProduct	KitDron, KitCamion...
Creator	CreadorKit
ConcreteCreator	CreadorDron, CreadorCamion...

```

Creator.Preparar()
    producto = CrearProducto()
    return producto.Usar()

ConcreteCreator
    override CrearProducto()
    return new ProductoConcreto()
  
```

Estructura mental

C# mínimo que demuestra el patrón

```

public interface IKit { string Revisar(); }

public abstract class CreadorKit {
    public abstract IKit CrearKit();
    public string Preparar() => CrearKit().Revisar();
}

public sealed class CreadorDron : CreadorKit {
    public override IKit CrearKit() => new KitDron();
}
  
```

Prueba práctica que debería pasar

- Guardar varios creadores en un arreglo de tipo CreadorKit[] y llamar Preparar() sin condicionales.
- Al agregar una moto: crear KitMoto y CreadorMoto; no tocar Preparar().

ERROR QUE BAJA PUNTAJE

Poner un gran switch dentro de Preparar() y llamarlo Factory Method. Si no hay subclases creadoras que redefinan la operación de creación, falta el mecanismo central.

3. Abstract Factory

PISTA DECISIVA

La variante elegida debe producir una familia completa de productos compatibles. El cliente elige la familia una vez y no mezcla sus componentes.

Cómo detectarlo

- Hay dos o más tipos de producto: factura, etiqueta, validador.
- Cada variante ofrece una versión de todos ellos: Europa, Latam, Asia.
- Mezclar variantes genera un problema explícito del dominio.

Roles en OmniExpress B

- AbstractFactory: IFabricaRegional.
- ConcreteFactories: Europa y Latam.
- AbstractProducts: factura, etiqueta y validador.
- Cliente: OperacionRegional.

La forma de la solución

Familia	Factura	Etiqueta	Validador
Europa	FacturaEuropa	EtiquetaEuropa	ValidadorEuropa
Latam	FacturaLatam	EtiquetaLatam	ValidadorLatam
Asia (extensión)	FacturaAsia	EtiquetaAsia	ValidadorAsia

C# mínimo que demuestra la familia

```
public interface IFabricaRegional {
    IFactura CrearFactura();
    IEtiqueta CrearEtiqueta();
    IValidadorPeso CrearValidador();
}

public sealed class FabricaEuropa : IFabricaRegional {
    public IFactura CrearFactura() => new FacturaEuropa();
    public IEtiqueta CrearEtiqueta() => new EtiquetaEuropa();
    public IValidadorPeso CrearValidador() => new ValidadorEuropa();
}
```

Qué debe hacer el cliente

```
public OperacionRegional(IFabricaRegional fabrica) {
    factura = fabrica.CrearFactura();
    etiqueta = fabrica.CrearEtiqueta();
    validador = fabrica.CrearValidador();
}
```

EXTENSIÓN TÍPICA

Agregar Asia requiere cuatro clases en este diseño: tres productos concretos y FabricaAsia. La lógica Procesar() no cambia; sí se registra la nueva fábrica en el punto de composición.

4. OmniExpress: resolver la consigna sin confundirse

Pregunta	Desafío A - kits	Desafío B - regiones
¿Qué varía?	Una variante de IKitSeguridad.	Tres tipos de producto a la vez.
Frase decisiva	Cada subclase decide qué kit crear.	Familia completa e interdependiente.
Patrón	Factory Method.	Abstract Factory.
Decisión	Elegir un creador concreto.	Elegir una fábrica regional.
Qué no debe cambiar	CreadorKit.Preparar().	OperacionRegional.Procesar().
Prueba	Cambiar creador cambia el kit.	Cambiar fábrica cambia los tres productos juntos.

Lectura guiada del enunciado

Texto de la consigna	Traducción a diseño
'interfaz de creación descentralizada'	La creación se reparte entre subclases creadoras.
'sin que la lógica principal conozca clases concretas'	Preparar usa IKitSeguridad.
'familia completa de productos interdependientes'	La fábrica expone un método por tipo de producto.
'mezclar regiones produce incumplimientos'	Todos los productos se obtienen de la misma fábrica.

Dilema que pueden pedirte justificar

¿POR QUÉ NO BASTAN FACTORY METHODS INDEPENDIENTES?

Porque cada creador podría seleccionar una región distinta. El problema no es solo instanciar objetos: es coordinar una familia. Abstract Factory agrupa las tres operaciones y hace que la región se elija una sola vez.

Checklist de entrega para este caso

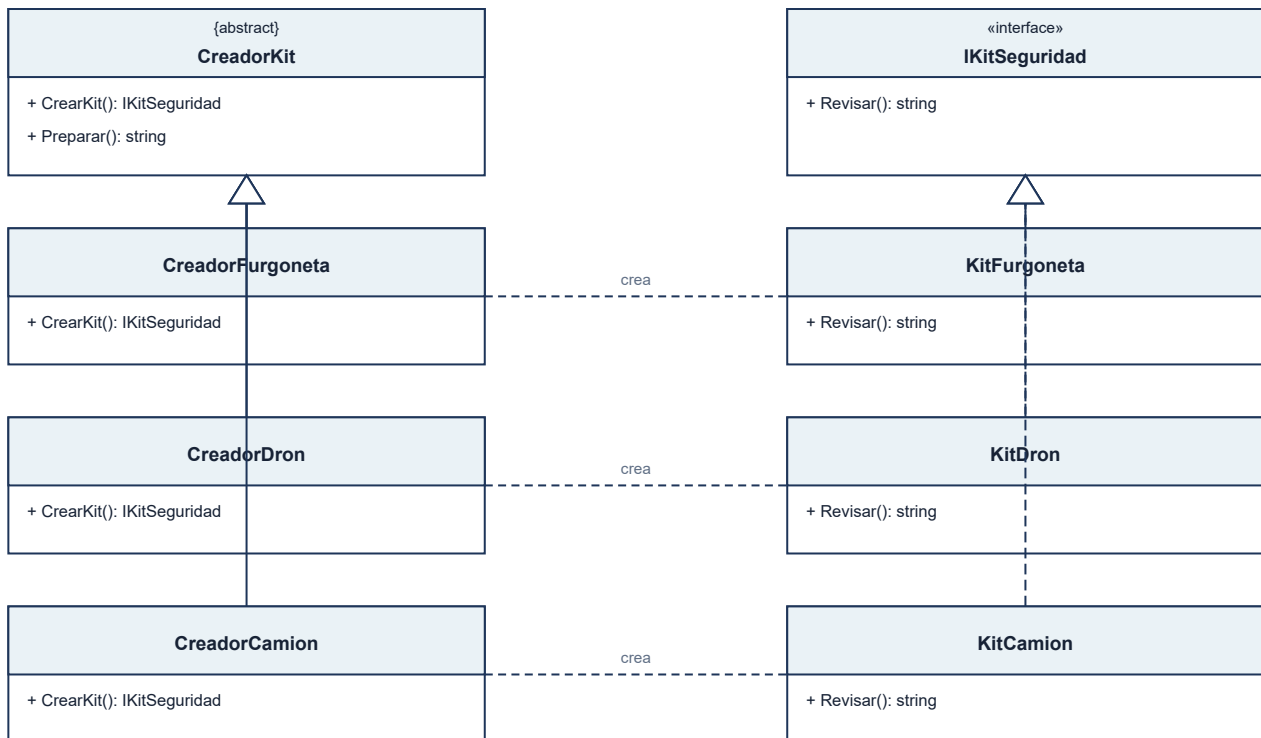
- UML A: Creator, ConcreteCreators, Product y ConcreteProducts; herencia y realizaciones correctas.
- UML B: fábrica abstracta, fábricas regionales, tres productos abstractos y seis concretos.
- Código: dos estructuras completas y coherentes con el UML.
- Main: usa CreadorKit[] e IFabricaRegional[] para mostrar polimorfismo.
- Justificación: nombra la frase decisiva, el mecanismo y el efecto observable.

TRAMPA PRINCIPAL

La palabra 'kit' puede sonar a familia, pero el requisito A lo modela como un único Product. En B sí hay tres tipos de producto que deben pertenecer a una misma región.

5. OmniExpress A: UML completo

Factory Method aplicado a los kits. El diagrama tiene que mostrar que la jerarquía de creadores decide qué producto concreto fabricar.



CÓMO LEER EL DIAGRAMA

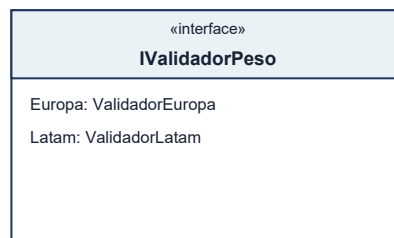
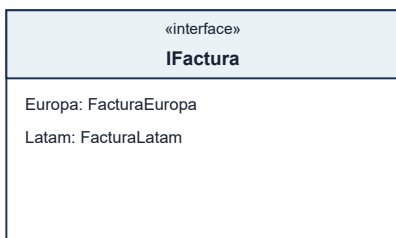
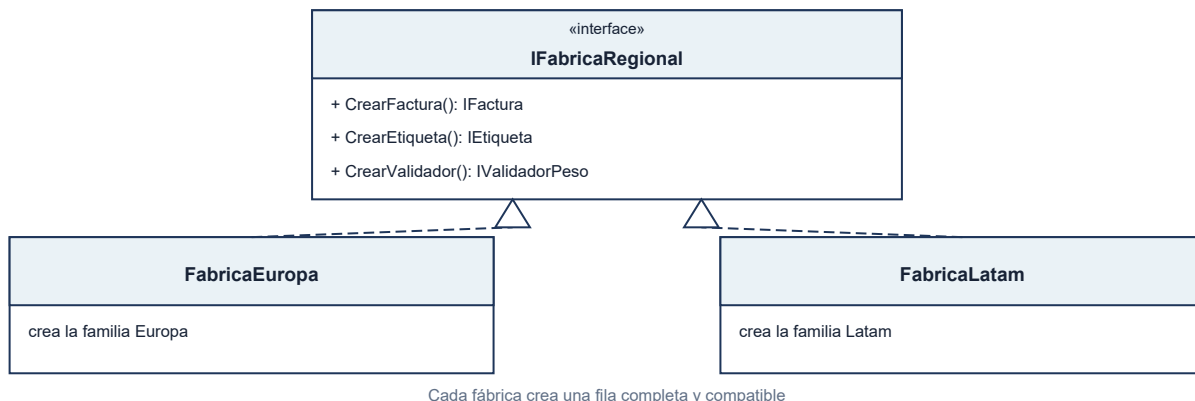
CreadorKit declara `CrearKit()` y conserva el flujo `Preparar()`. Cada creador concreto redefine la creación y devuelve un objeto mediante `IKitSeguridad`.

Qué debe coincidir con el código

- Las flechas de generalización apuntan a CreadorKit.
- Las realizaciones de los productos apuntan a IKitSeguridad.
- Preparar usa el contrato y no contiene un switch por tipo de vehículo.
- Para agregar Moto se suman KitMoto y CreadorMoto; Preparar no cambia.

6. OmniExpress B: UML completo

Abstract Factory aplicado a familias regionales. La región se elige una vez y sus tres productos se crean juntos.



Familia	Factura	Etiqueta	Validador
Europa	FacturaEuropa	EtiquetaEuropa	ValidadorEuropa
Latam	FacturaLatam	EtiquetaLatam	ValidadorLatam
Asia	FacturaAsia	EtiquetaAsia	ValidadorAsia

Cliente y comprobaciones

- OperacionRegional depende de IFabricaRegional y guarda IFactura, IEtiqueta e IValidadorPeso.
- Los tres productos se piden a la misma fábrica; opcionalmente se comprueba que sus identificadores de región coincidan.
- Agregar Asia suma cuatro clases: tres productos concretos y FabricaAsia. Procesar no cambia.
- Agregar un nuevo tipo de producto sí obliga a ampliar IFabricaRegional y todas sus fábricas.

MAIN QUE DEMUESTRA LA DIFERENCIA

Usá un arreglo CreadorKit[] para Factory Method y otro IFabricaRegional[] para Abstract Factory. El primero cambia un kit; el segundo cambia una familia completa.

7. Singleton

PISTA DECISIVA

La consigna exige una y solo una instancia dentro de la aplicación y un punto de acceso global controlado.

Señales en el enunciado

- Configuración global única.
- Evitar copias desactualizadas entre sí.
- Mismo objeto cada vez que se accede.
- Control de creación: el cliente no puede hacer new.

Elementos que deben aparecer

- Constructor privado.
- Campo o inicializador estático controlado.
- Propiedad pública de acceso.
- Clase sellada si querés impedir otra vía de instanciación por herencia.

C# mínimo y seguro para la inicialización

```
public sealed class ConfiguracionParque {
    private static readonly Lazy<ConfiguracionParque> instancia =
        new(() => new ConfiguracionParque());

    private ConfiguracionParque() { }
    public static ConfiguracionParque Instancia => instancia.Value;
}
```

Qué probar en Main

```
var a = ConfiguracionParque.Instancia;
var b = ConfiguracionParque.Instancia;
Console.WriteLine(ReferenceEquals(a, b)); // True
```

UNICIDAD NO ES LO MISMO QUE CONCURRENCIA

Lazy<T> puede proteger la inicialización, pero el estado mutable posterior necesita su propia estrategia, por ejemplo un lock alrededor de lecturas y actualizaciones relacionadas.

Cómo justificarlo en Wonderland

'Elijo Singleton porque toda la aplicación debe compartir una sola configuración del parque.

ConfiguracionParque oculta el constructor y expone Instancia; por eso dos clientes obtienen la misma referencia y no mantienen copias divergentes.'

No lo uses cuando...

- Solo necesitás compartir un objeto, pero podrían existir varias instancias válidas.
- La consigna no pide unicidad y alcanza con inyectar una instancia creada en el punto de composición.

8. Composite

PISTA DECISIVA

Existe una estructura en árbol y el cliente debe tratar de la misma forma a un elemento individual y a un grupo completo.

Roles

Rol	Wonderland
Component	IElementoParque
Leaf	Atraccion
Composite	ZonaCompuesta
Children	List

Operación uniforme

- La hoja devuelve su propio valor.
- El compuesto recorre hijos y acumula.
- Un hijo puede ser hoja u otro compuesto.
- El cliente llama siempre al mismo contrato.

C# mínimo que demuestra la recursividad

```
public interface IElementoParque {
    double CalcularConsumo();
}

public sealed class Atraccion : IElementoParque {
    public double CalcularConsumo() => consumo;
}

public sealed class ZonaCompuesta : IElementoParque {
    private readonly List<IElementoParque> hijos = new();
    public void Agregar(IElementoParque e) => hijos.Add(e);
    public double CalcularConsumo() =>
        hijos.Sum(h => h.CalcularConsumo());
}
```

Leer la consigna como un árbol

Nivel	Ejemplo	Resultado de CalcularConsumo
Hoja	Montaña rusa	120
Compuesto	Área mecánica = montaña + barco	120 + 65 = 185
Compuesto	Zona aventura = área	185
Raíz	Parque = zona + carrusel	185 + 30 = 215

ERROR QUE BAJA PUNTAJE

Usar una lista de atracciones pero impedir que una zona contenga otra zona. Eso no resuelve el requisito recursivo. También es incorrecto que el cliente pregunte el tipo de cada hijo para decidir cómo calcular.

9. Observer

PISTA DECISIVA

Cuando un objeto cambia, debe notificar automáticamente a varios receptores registrados sin depender de sus clases concretas.

Participantes

- Subject: Atraccion.
- Observer: IObservador.
- ConcreteObservers: panel, mantenimiento y pantalla.
- Evento: CambiarEstado.

Métodos esperables

- Suscribirse(observer).
- Desuscribirse(observer).
- CambiarEstado(nuevoEstado).
- Actualizar(datosDelCambio).

C# mínimo que demuestra el patrón

```
public interface IObservador {
    void Actualizar(string nombre, EstadoAtraccion estado);
}

public sealed class Atraccion {
    private readonly List<IObservador> observadores = new();
    public void Suscribirse(IObservador o) => observadores.Add(o);
    public void Desuscribirse(IObservador o) => observadores.Remove(o);

    public void CambiarEstado(EstadoAtraccion nuevo) {
        if (Estado == nuevo) return;
        Estado = nuevo;
        foreach (var o in observadores.ToArray())
            o.Actualizar(Nombre, Estado);
    }
}
```

Prueba práctica

```
montana.Suscribir(new PanelOperaciones());
montana.Suscribir(new AlertaMantenimiento());
montana.Suscribir(new PantallaPublica());
montana.CambiarEstado(EstadoAtraccion.FueraDeServicio);
// Se producen tres reacciones.
```

EXTENSIÓN TÍPICA

Si aparece una app móvil, crear AppMovil : IObservador y suscribirla. Atraccion no cambia porque solo conoce el contrato.

Descartes

- No es Observer solo porque existan muchos objetos; debe haber un cambio y notificación.
- El sujeto no debería crear directamente las ventanas o receptores concretos.
- Observer no decide por sí solo asincronía, reintentos ni qué hacer si un receptor falla.

10. Wonderland: separar tres problemas en una sola solución

Fragmento de la consigna	Patrón	Traducción concreta
'una y solo una instancia'	Singleton	ConfiguracionParque con constructor privado e Instancia.
'se organiza como un árbol' y 'misma interfaz'	Composite	Atraccion y ZonaCompuesta implementan IElementoParque.
'notificar automáticamente a varias dependencias'	Observer	Atraccion conserva IObservador y notifica en CambiarEstado.

Cómo integrar sin mezclar responsabilidades

ATRACCION TIENE DOS ROLES

Es Leaf de Composite porque devuelve sus métricas, y es Subject de Observer porque su cambio de estado dispara avisos.

ZONACOMPUESTA TIENE UN ROL

Es Composite: contiene IElementoParque y suma resultados. No necesita saber de observadores.

Main que evidencia los tres patrones

```
var c1 = ConfiguracionParque.Instancia;
var c2 = ConfiguracionParque.Instancia;
Console.WriteLine(ReferenceEquals(c1, c2)); // Singleton

var parque = new ZonaCompuesta("Parque");
parque.Agregar(zona); parque.Agregar(carrusel);
Console.WriteLine(parque.CalcularConsumo()); // Composite: 215

montana.Suscribir(new PanelOperaciones());
montana.Suscribir(new AlertaMantenimiento());
montana.Suscribir(new PantallaPublica());
montana.CambiarEstado(EstadoAtraccion.FueraDeServicio); // Observer
```

Respuestas cortas que suelen pedir

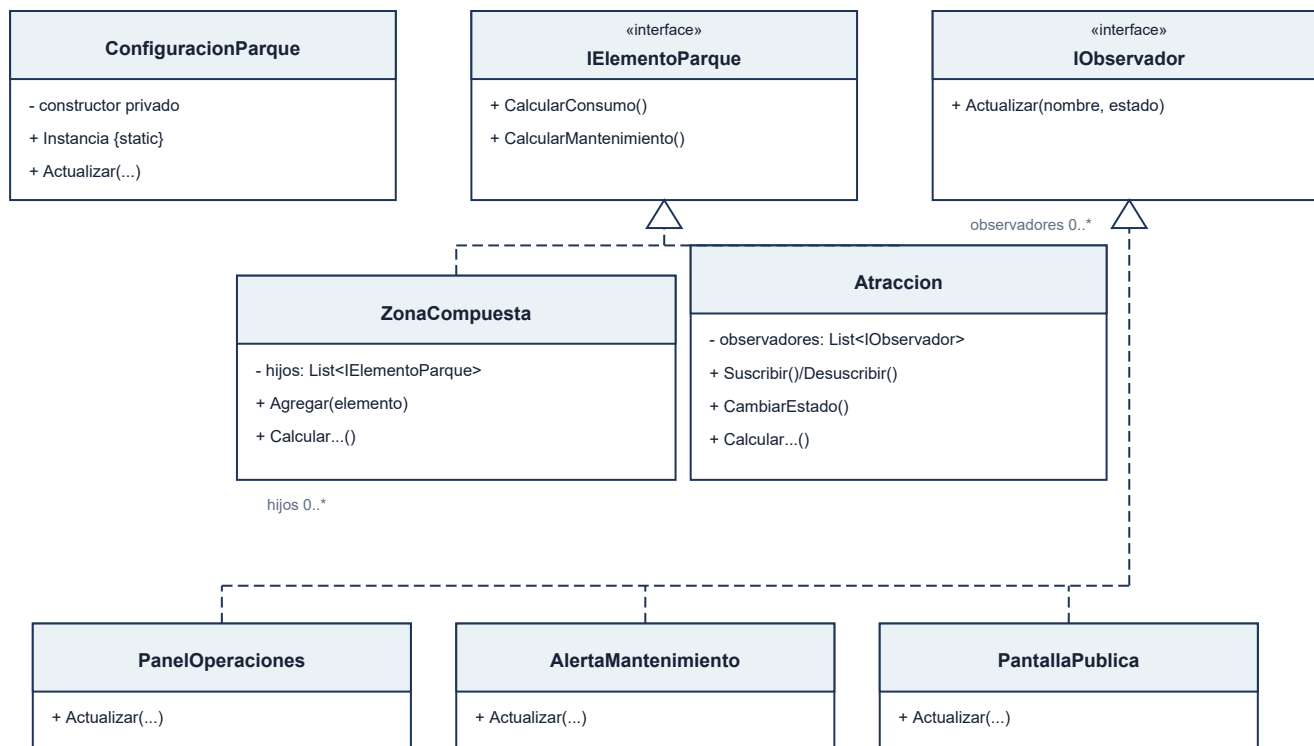
Pregunta	Respuesta útil
¿Qué evita la instancia única?	Copias de configuración que podrían quedar desactualizadas entre sí.
¿Qué simplifica Composite?	El cliente calcula métricas sin preguntar si recibió una atracción o una zona.
¿Qué pasa al agregar una app móvil?	Se implementa IObservador y se registra; la lógica de Atraccion no cambia.

CHEQUEO FINAL

El UML, el código y el Main deben mostrar los mismos nombres, contratos, relaciones y comportamientos. Si uno cuenta una historia distinta, la solución no está cerrada.

11. Wonderland: UML integrado

Los tres patrones conviven, pero cada uno resuelve una necesidad distinta. Atraccion es el punto de unión: hoja de Composite y sujeto de Observer.



COMPOSITE

ZonaCompuesta mantiene 0..* hijos del tipo IElementoParque. Una atracción devuelve su valor; una zona suma recursivamente.

OBSERVER

Atraccion mantiene 0..* observadores. Al cambiar el estado, recorre una copia de la lista y llama Actualizar().

Controles prácticos que aparecen en la resolución

Parte	Qué comprobar
Singleton	Lazy controla la inicialización; un lock protege lecturas y actualizaciones del estado mutable.
Composite	Una zona vacía devuelve cero; conviene impedir ciclos y dobles conteos si la edición es libre.
Observer	No notificar si el estado no cambió; ToArray permite que un callback modifique suscripciones.
Main	Misma referencia = True; consumo = 215; mantenimiento = 8500; tres avisos al cambiar Titan.

NO INVENTAR REGLAS

El enunciado no dice que una atracción fuera de servicio pase a consumir cero. Tampoco define normativa regional real. Implementará y explicará solo las reglas pedidas.

12. UML para resolver los prácticos

Clases, flechas y relaciones: cómo dibujarlas y cómo leerlas sin confusiones.

Cómo se representa una clase

Pedido
- numero: int - total: decimal
+ Confirmar(): void + CalcularTotal(): decimal

TRES COMPARTIMENTOS

Nombre, atributos y operaciones.
 + significa público; - significa privado.
 Un miembro subrayado o marcado {static} pertenece al tipo.
 La cursiva o {abstract} indica un elemento abstracto.

Relaciones que tenés que reconocer

Asociación navegable	A		B
Dependencia	A		B
Generalización	A		B
Realización de interfaz	A		B
Agregación compartida	A		B
Composición	A		B

Relación	Lectura práctica
Asociación navegable	A conoce o puede acceder a B.
Dependencia	A usa a B de manera puntual; se dibuja con línea discontinua.
Generalización	A hereda de B; el triángulo apunta a la clase base B.
Realización de interfaz	A implementa B; línea discontinua y triángulo hacia la interfaz B.
Agregación compartida	A es el todo, pero la parte B puede existir independientemente.
Composición	A es el todo y controla fuertemente el ciclo de vida de la parte B.

REGLA VISUAL INDISPENSABLE

En agregación y composición, el rombo va en el todo. En generalización y realización, el triángulo apunta a la base o interfaz.

13. UML: multiplicidades y aplicación

Notación que puede aparecer en los diagramas del parcial y qué significa desde el punto de vista del objeto relacionado.

Notación	Cómo leerla
1	Exactamente uno.
0..1	Ninguno o uno.
0..*	Cero o muchos.
1..*	Al menos uno.
«interface»	Contrato que implementan clases.
{abstract}	Clase u operación abstracta.
{static} o subrayado	Miembro que pertenece al tipo, no a cada instancia.

CÓMO INTERPRETAR LA MULTIPLICIDAD

La multiplicidad se lee desde una instancia del extremo opuesto. Si junto a Observador aparece 0..*, significa que un Subject puede tener cero o muchos observadores.

Qué tiene que verse en cada patrón

Patrón	Elementos mínimos en el diagrama
Factory Method	Creator abstracto, método de creación, ConcreteCreators, Product y ConcreteProducts.
Abstract Factory	Fábrica abstracta, fábricas concretas, un contrato por tipo de producto y variantes por familia.
Singleton	Constructor privado, instancia estática y acceso público estático.
Composite	Component común, Leaf, Composite y asociación 0..* hacia Component.
Observer	Subject con 0..* Observer, métodos de alta/baja y operación de actualización.

Errores concretos que te pueden bajar puntaje

- Dibujar herencia como realización o invertir la dirección del triángulo.
- Poner el rombo en la parte en lugar del todo.
- Usar composición fuerte solo porque una clase tiene un campo o una lista.
- Confundir el patrón Composite con la relación UML de composición.
- Mostrar relaciones o firmas que después no existen en el código.

ORDEN RECOMENDADO PARA DIBUJAR

1. Contratos. 2. Clases concretas. 3. Herencia o realización. 4. Asociaciones y multiplicidades. 5. Métodos clave. Después verificá que las firmas coincidan con el código.

14. Entrenamiento: detectar por la consigna

Tapá la última columna, respondé en voz alta y justificá con la señal decisiva.

Enunciado	Respuesta y pista
Cada subclase de logística elige el transporte que usa el mismo flujo de entrega.	Factory Method: creación redefinida por subclases.
Un tema visual crea botón, menú y ventana que deben tener el mismo estilo.	Abstract Factory: familia de productos compatibles.
Grupos de figuras contienen figuras y otros grupos; todos responden a Mover.	Composite: árbol más operación uniforme.
Un sensor avisa a alarmas registradas cuando cambia la temperatura.	Observer: evento más receptores suscriptos.
Se exige un único objeto de configuración por proceso y acceso controlado.	Singleton: unicidad explícita.
Una factura guarda una lista de renglones, pero no hay grupos recursivos.	No alcanza para Composite: es una colección común.
Factura, etiqueta y validador eligen su región por separado.	Riesgo de mezcla: conviene Abstract Factory.
Hay miles de registros de usuarios.	No alcanza para Observer: falta un evento y notificación.

Cinco consignas nuevas para practicar

Caso	Qué deberías detectar
Un editor exporta PDF o imagen; cada editor especializado decide el exportador, pero Guardar() no cambia.	Factory Method.
Un juego elige una facción y debe crear soldado, edificio y vehículo de esa misma facción.	Abstract Factory.
Carpetas contienen archivos y otras carpetas; ambos informan tamaño mediante el mismo método.	Composite.
Una orden cambia de estado y avisa a stock, facturación y envíos registrados.	Observer.
Toda la aplicación usa una única caché de configuración y no permite construcción externa.	Singleton.

CÓMO AUTOCORREGIRTE

Tu respuesta debe contener: necesidad de la consigna + mecanismo del patrón + participantes con nombres del caso + una prueba observable en Main.

15. Preguntas relámpago de repaso

Respondelas sin mirar la segunda columna. Si dudás, reconstruí el mecanismo y sus participantes.

Pregunta	Respuesta corta
1. ¿Qué distingue Factory Method de Abstract Factory?	FM delega un punto de creación a subclases; AF coordina una familia de productos relacionados.
2. ¿Por qué el kit de OmniExpress A no obliga a usar AF?	Porque el requisito lo trata como un solo Product cuya variante decide cada ConcreteCreator.
3. ¿Qué se agrega para incorporar Asia?	FacturaAsia, EtiquetaAsia, ValidadorAsia y FabricaAsia; se actualiza la selección, no Procesar.
4. ¿Qué dos roles cumple Atraccion?	Leaf de Composite y Subject de Observer.
5. ¿Cómo calcula Composite un total?	La hoja devuelve su valor; el compuesto suma recursivamente los resultados de sus hijos.
6. ¿Cómo se agrega una pantalla móvil?	Implementa IObservador y se suscribe; Atraccion no cambia su lógica.
7. ¿Lazy vuelve seguro todo el estado del Singleton?	No. Protege la inicialización; el estado mutable necesita sincronización aparte.
8. ¿Hacia dónde apunta el triángulo UML?	Hacia la clase base en generalización y hacia la interfaz en realización.
9. ¿Dónde va el rombo?	En el extremo que representa el todo, no en la parte.
10. ¿Cómo comprobás que la solución es coherente?	Los mismos contratos, nombres, relaciones y comportamientos aparecen en UML, código, Main y justificación.

RESPUESTA MÍNIMA QUE SUMA PUNTOS

Nombrá la pista del enunciado, el patrón, los roles con nombres del caso y una evidencia observable del Main. Evitá responder solamente 'porque desacopla'.

16. Procedimiento de examen

Tiempo	Acción	Resultado que deberías tener
0-15 min	Leer todo, separar desafíos y subrayar restricciones.	Una frase decisiva por problema.
15-35 min	Asignar roles y dibujar UML mínimo.	Contratos, clases, relaciones y firmas.
35-90 min	Implementar desde interfaces hacia concretas y Main.	Código compilable y demostración.
90-110 min	Probar variantes y resultados.	Evidencia de cada mecanismo.
110-120 min	Escribir justificaciones y revisar coherencia.	UML, código y explicación alineados.

Plantilla de resolución para cualquier ejercicio

Paso	Qué escribir
1. Necesidad	La restricción exacta que aparece en la consigna.
2. Patrón	Nombre y una oración que conecte necesidad con mecanismo.
3. Participantes	Interfaz/clase base, concretas, cliente y relaciones.
4. UML	Mismos nombres y firmas que tendrá el código.
5. Código	Primero contratos; luego concretas; al final el cliente.
6. Main	Una acción que haga visible el comportamiento del patrón.
7. Extensión	Qué clases se agregan y qué parte estable no cambia.

Frases listas para adaptar

ELECCIÓN

"Elijo [patrón] porque el enunciado exige [restricción]. El patrón resuelve esto mediante [mecanismo]."

APLICACIÓN

"[Clase] actúa como [rol]; [otra clase] implementa [contrato]. En Main se comprueba que [resultado]."

EXTENSIÓN

"Para agregar [variante] se crean [clases]. La lógica [estable] no cambia; solo se actualiza el punto donde se seleccionan o registran implementaciones."

Último control antes de entregar

- Compila y el Main demuestra exactamente lo pedido.
- Las interfaces del UML coinciden con las del código.
- No agregaste reglas de negocio que la consigna nunca definió.
- La justificación explica por qué, no solo nombra el patrón.